

À la fréquence de Fourier

1.1 La Transformée de Fourier

À quoi ça sert ?

Un monde parallèle où les choses deviennent plus simples (ou plus compliquées)

- **1.2.1 Traitement du signal** Séparation des contributions de chaque fréquence
- **1.2.2 Avantage computationnel** L'opération de convolution devient une multiplication classique

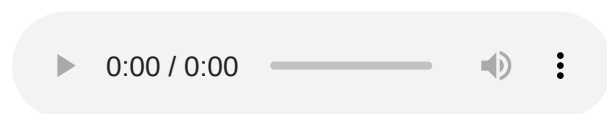
1.1.1 Illustration : séparation des fréquences

Audio

la



ré



la + ré



Signal temporel ↔

Un son est un signal continu mais on va l'échantillonner. C'est à dire qu'on va prendre un nombre fini de valeur par seconde à des distance égale dans le temps. Dans cet exemple, on prend une valeur toute les 1024^e de seconde.

Fréquence d'échantillonnage =  1024

Ça correspond à une distance en seconde entre deux échantillons de:

```
 $\Delta t = 0.0009765625$ 
```

```
1  $\Delta t = 1 / \text{échantillonnage}$ 
```

nombre_échantillons =  1024

```
temps_total = 1.0
```

```
1 temps_total = nombre_échantillons *  $\Delta t$ 
```

En prenant 1024, le signal dure 1.0 secondes. Comme ces nombres sont équidistants (distance de Δt secondes entre eux), on peut représenter cette suite de façon compact comme suit:

```
temps = 0.0009765625:0.0009765625:1.0
```

```
1 temps = range( $\Delta t$ , stop=temps_total, length=nombre_échantillons)
```

Le *la* est une note de musique de fréquence 440 Hz

```
la =
```

```
► [-0.903989+0.427555im, 0.634393-0.77301im, -0.24298+0.970031im, -0.19509-0.980785im, 0.5]
```

```
1 la = cispi.(2*440*temps)
```

La note de musique *ré* a une fréquence de 293.7 Hz

```
ré =
```

```
► [-0.229267+0.973364im, -0.894874-0.44632im, 0.639596-0.768711im, 0.601597+0.7988im, -0.9]
```

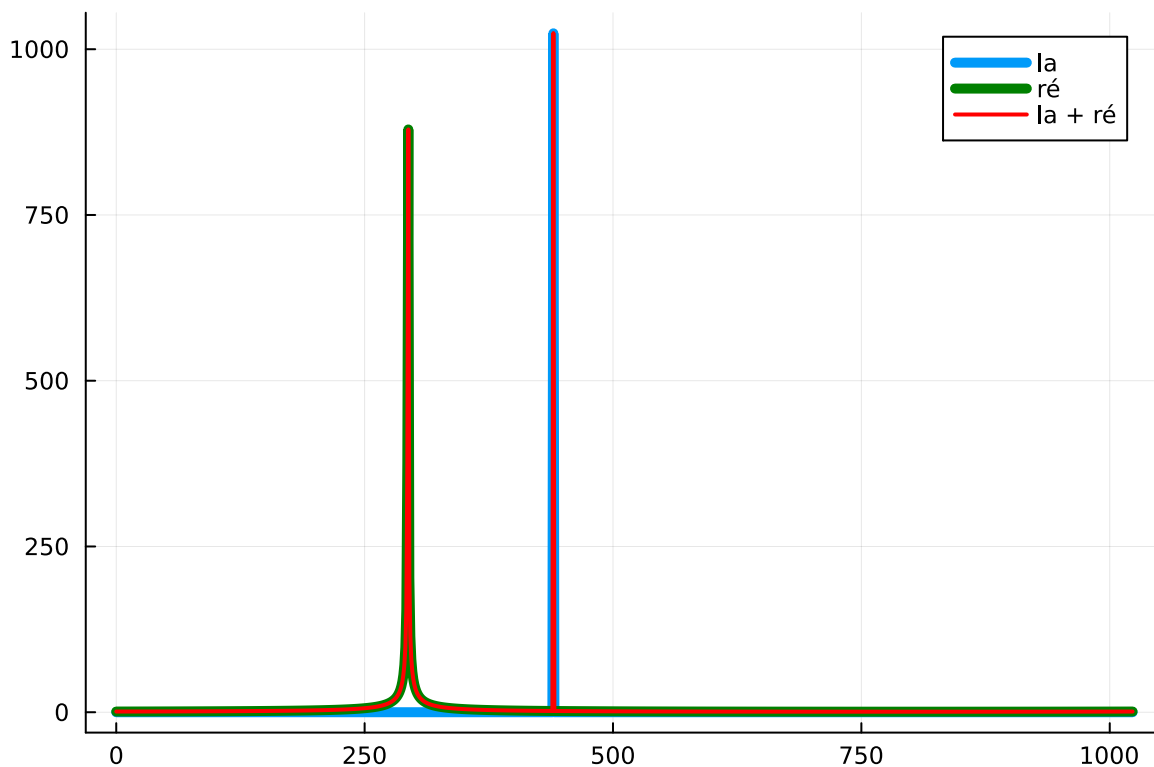
```
1 ré = cispi.(2*293.7*temps)
```

temps_zoom =  0.032447076612903226



Signal fréquentiel ⇔

abs ▼



On a la valeur de la transformée de fourier, tous les 1.0 Hz

1.1.2 Calcul rapide de convolution ⇔

La convolution continue:

$$(f * g)(x) = \int_{-\infty}^{\infty} f(s)g(x-s)ds$$

La convolution discrète:

$$(f * g)_n = \sum_{k=-\infty}^{\infty} f_k g_{n-k}$$

Propriétés algébriques:

commutatif	$f * g = g * f$
associatif	$(f * g) * h = f * (g * h)$
distributif	$f * (g + h) = f * g + f * h$
conjugué	$\overline{f * g} = \bar{f} * \bar{g}$

La cross-corrélation:

$$(f \star g)(x) = \int_{-\infty}^{\infty} f(s)g(x+s)ds \quad (f \star g)_n = \sum_{-\infty}^{\infty} f_k g_{n+k} ds$$

Convolution theorem: Notons la transformée de Fourier de f par $\mathcal{F}(f)$. Par la transformée de Fourier, la convolution devient un produit classique:

$$f * g = \mathcal{F}^{-1}(\mathcal{F}(f) \cdot \mathcal{F}(g))$$

► Quel est la complexité de la convolution vs produit classique s'ils sont discrets de longueur n ?

1.1.2.1 Illustration: convolution d'images ⇔

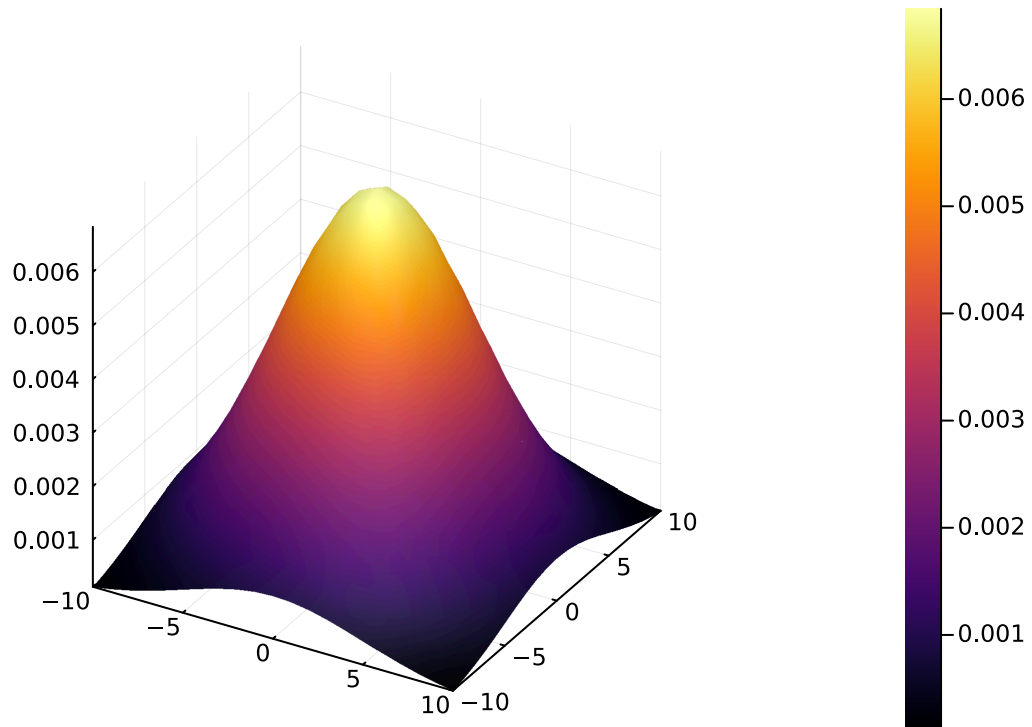
mandrill =



```
1 mandrill = testimage("mandrill")
```

Gaussian Kernel ↻

La fonction `gaussian(d)` donne une matrice de taille $(2d + 1) \times (2d + 1)$ contenant la valeur d'une Gaussienne.



```
1 surface(-2d:2d, -2d:2d, Kernel.gaussian(d))
```

d = 5

La fonction `imfilter` calcule la cross-corrélation.



```
1 imfilter(mandrill, Kernel.gaussian(d))
```

La convolution est obtenue avec `reflect`.



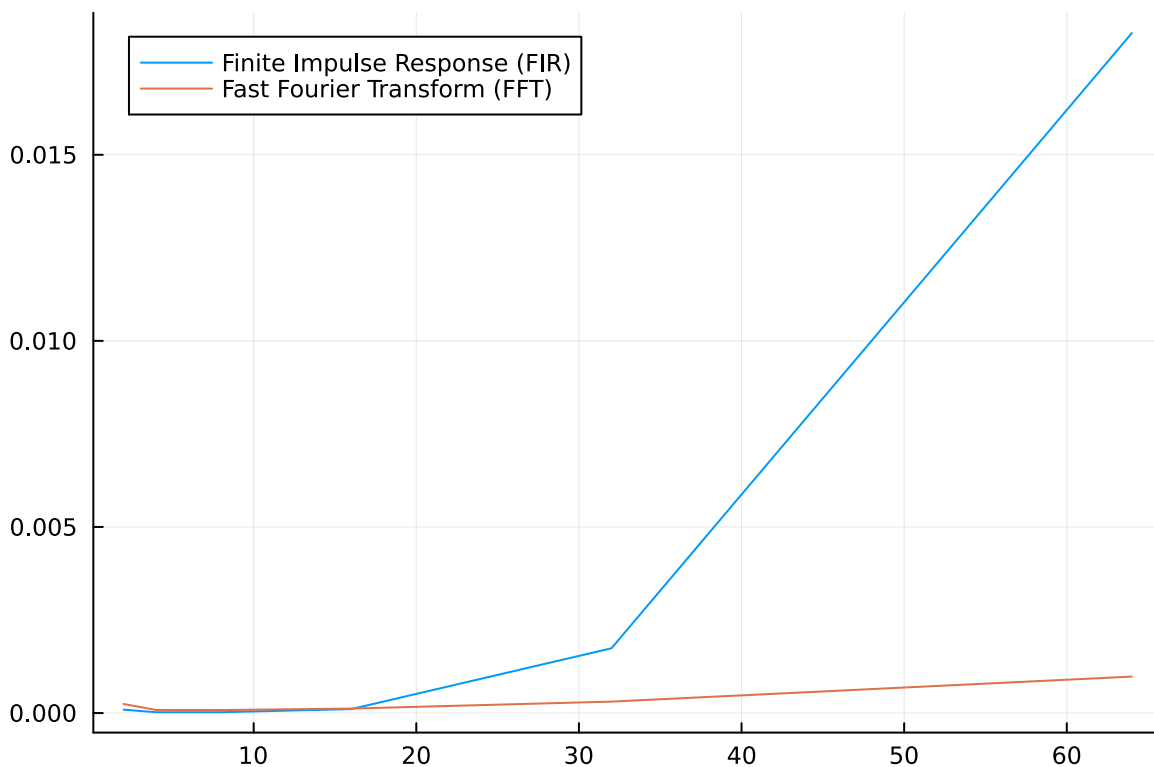
```
1 imfilter(mandrill, reflect(Kernel.gaussian(d)))
```

► Pourquoi obtient-on le même résultat avec la convolution et la cross-corrélation ?

Performance ⇔

imfilter_time (generic function with 1 method)

```
1 function imfilter_time(n, alg)
2     A = rand(n, n)
3     B = rand(n, n)
4     return @elapsed imfilter(A, B, alg)
5 end
```



```

1 let
2   n = 2 .^ (1:6)
3   fir_time(n) = imfilter_time(n, Algorithm.FIR())
4   fft_time(n) = imfilter_time(n, Algorithm.FFT())
5   plot(n, fir_time ◦ Int, label = "Finite Impulse Response (FIR)")
6   plot!(n, fft_time ◦ Int, label = "Fast Fourier Transform (FFT)")
7 end

```

```

3x3 OffsetArray(::Matrix{Int64}, -1:1, -1:1) with eltype Int64 with indices -1:1×-1:1:
 0  1  0
 1 -4  1
 0  1  0

```

```

1 convert(AbstractArray, Kernel.Laplacian())

```



Quelle est la complexité de FIR ? et de de FFT ?

Autre kernels ⇌



```
1 imfilter(mandrill, Kernel.Laplacian())
```

Différents kernels permettent d'analyser des aspects différents d'une image. Le kernel à utilisé peut aussi être **appris**, c'est la base des **Convolutional Neural Networks** (CNNs) !

1.1.2.2 Illustration : Le produit de polynômes ⇄

```
deg = 4
```

```
1 deg = 4
```

```
p = 0.9490706240067519 + 0.1920142317450696·x + 0.038371286224697654·x2 +  
0.5774959630156685·x3
```

```
1 p = Polynomial(rand(deg))
```

```
q = 0.4028871024231774 + 0.07284974639558217·x + 0.4828233556291758·x2 + 0.5814126083328487·x3
```

```
1 q = Polynomial(rand(deg))
```

$0.38236831370103713 + 0.14649961172217238 \cdot x + 0.4876809478223055 \cdot x^2 + 0.8799715963663693 \cdot x^3 + 0.17223648294090904 \cdot x^4 + 0.3011380883345152 \cdot x^5 + 0.33576343415863014 \cdot x^6$

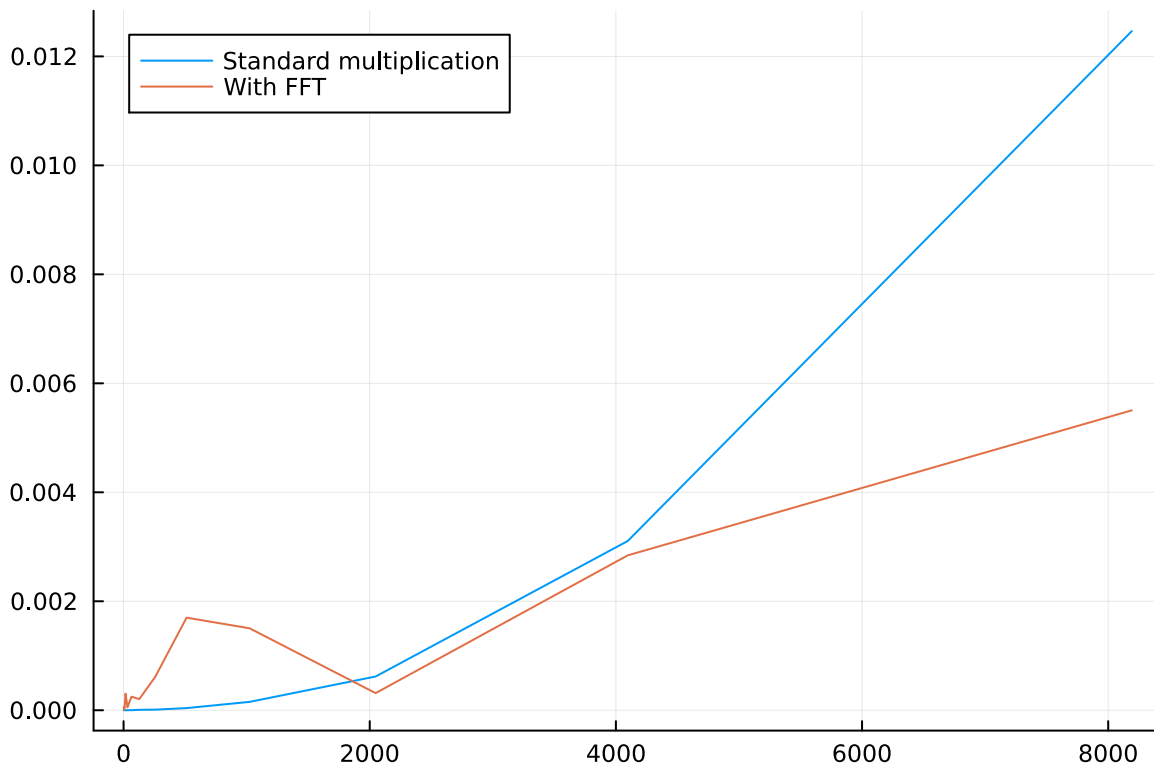
```
1 p * q
```

$0.3823683137010371 + 0.14649961172217243 \cdot x + 0.4876809478223054 \cdot x^2 + 0.8799715963663692 \cdot x^3 + 0.17223648294090912 \cdot x^4 + 0.3011380883345152 \cdot x^5 + 0.3357634341586302 \cdot x^6$

```
1 Polynomials.poly_multiplication_fft(p, q)
```

poly_time (generic function with 1 method)

```
1 function poly_time(n, with_fft::Bool)
2     p = Polynomial(rand(n))
3     q = Polynomial(rand(n))
4     @elapsed if with_fft
5         Polynomials.poly_multiplication_fft(p, q)
6     else
7         p * q
8     end
9 end
```



```

1 let
2   n = 2 .^ (1:13)
3   std_time(n) = poly_time(n, false)
4   fft_time(n) = poly_time(n, true)
5   plot(n, std_time ◦ Int, label = "Standard multiplication")
6   plot!(n, fft_time ◦ Int, label = "With FFT")
7 end

```



Quelle est la complexité du produit de polynômes avec ou sans la FFT ?



Comment utiliser cela pour calculer le produit de grand nombres ?

1.1.2 Définition de la transformée de Fourier

La transformée de Fourier continue et son inverse:

$$F(\xi) = \int_{-\infty}^{\infty} f(t) e^{-i2\pi\xi t} dt$$

$$f(t) = \int_{-\infty}^{\infty} F(\xi) e^{i2\pi\xi t} d\xi$$

La transformée de Fourier discrète et son inverse:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi \frac{k}{N} n}$$

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{i2\pi \frac{k}{N} n}$$

Intuition Le signal pur "la" est $f_{1a}(t) = \exp(2\pi 440ti) = \cos(2\pi 440t) + i \sin(2\pi 440t)$. Si j'évalue en $t = 1/440s$, j'obtiens la fin d'une période: $\exp(2\pi i) = \cos(2\pi) + i \sin(2\pi)$. Ces exponentielles pures sont *orthogonales*, c'est à dire que ces intégrales entre fréquences différentes sont nulles.

$$F(440) = \int_{-\infty}^{\infty} f(t) e^{-i2\pi 440t} dt = \int_{-\infty}^{\infty} 1 dt = \infty$$

$$F(293.7) = \int_{-\infty}^{\infty} f(t) e^{-i2\pi 293.7t} dt = 0$$

Cet intégrale permet donc de détecter la quantité d'une certaine fréquence dans un signal, en ignorant toutes les autres fréquences!

Rajouter linearité : TODO

► **Que vaut la distance entre deux valeurs successive en temporel et en fréquentiel pour les signaux discrets ?**

TODO : même dimension en temporel et fréquentiel

► **Que vaut le signal en dehors de ses N points ?**

1.1.3 Le replis spectral ⇔

On vient de voir que le signal est extrapolé périodiquement par la transformée de fourier discrète. Si on prend un signal de 0 à $t_{\max}$ et qu'on divise la fréquence d'échantillonnage de sa transformée de fourier par 2. Ça signifie que le signal sera 2 fois moins long.

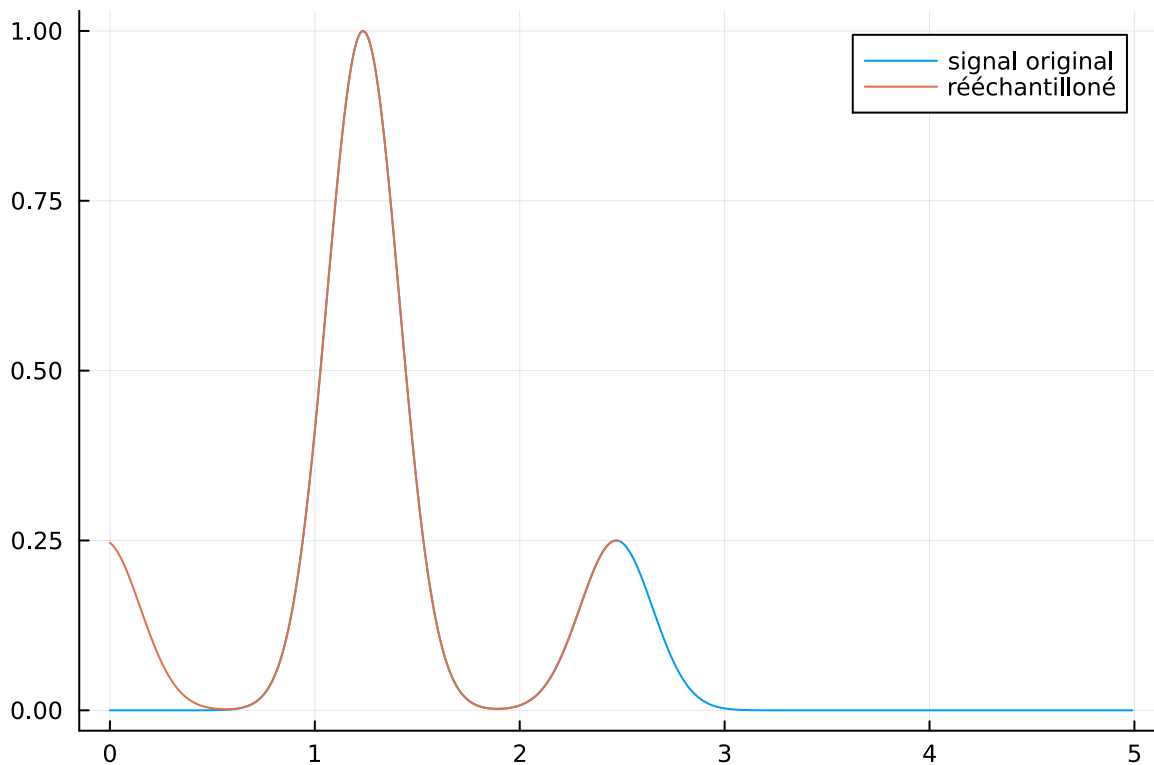
► **Que va-t-il advenir de la deuxième moitié du signal ?**

Illustrons cela avec la fonction $\exp(-(t-\text{centre})^2)$ qui est initialement échantillonnée avec 512 échantillons de 0 à $t_{\max}$ et qu'on ré-échantillonne ensuite à 512 / ratio en fréquentiel, ce qui divise $t_{\max}$ par ratio également.

t_max = 5.0

centre = 2.470967741935484

ratio = 2



1.2 Fast Fourier Transform ⇔

Attardons-nous à présent sur l'algorithme utilisé pour calculer la DFT (transformée de Fourier discrète). On a vu précédemment que cet algorithme a une complexité de $\Omega(n \log(n))$. C'est parmi le top 10 des meilleurs algorithmes du 20^e siècle ! Son Implémentation la plus rapide est dans la librairie FFTW : the Fastest Fourier Transform in the West ! Comment est-elle la plus rapide ? Voir ici. L'algorithme est initialement découvert par Gauss en 1805 puis redécouvert par Cooley et Tukey en 1965.

La DFT est en fait une évaluation d'un polynôme aux différentes racines N-èmes de l'unité:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi \frac{k}{N} n}$$

$$X_k = \sum_{n=0}^{N-1} x_n z_{k,N}^n \quad z_{k,N} = e^{-i2\pi \frac{k}{N}}$$

La DFT peut aussi être vue comme un produit matriciel:

$$X = \begin{bmatrix} 1 & 1 & \dots & 1 & 1 \\ 1 & z_N & \dots & z_N^{N-2} & z_N^{N-1} \\ 1 & z_N^2 & \dots & z_N^{2(N-2)} & z_N^{2(N-1)} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & z_N^{(N-1)2} & \dots & z_N^{(N-1)(N-2)} & z_N^{(N-1)(N-1)} \end{bmatrix} x$$

Un produit matriciel a complexité $\Omega(n^2)$. Ça ne nous donne pas une complexité de $\Omega(n \log n)$, il va falloir utiliser la structure assez spéciale de cette matrice...

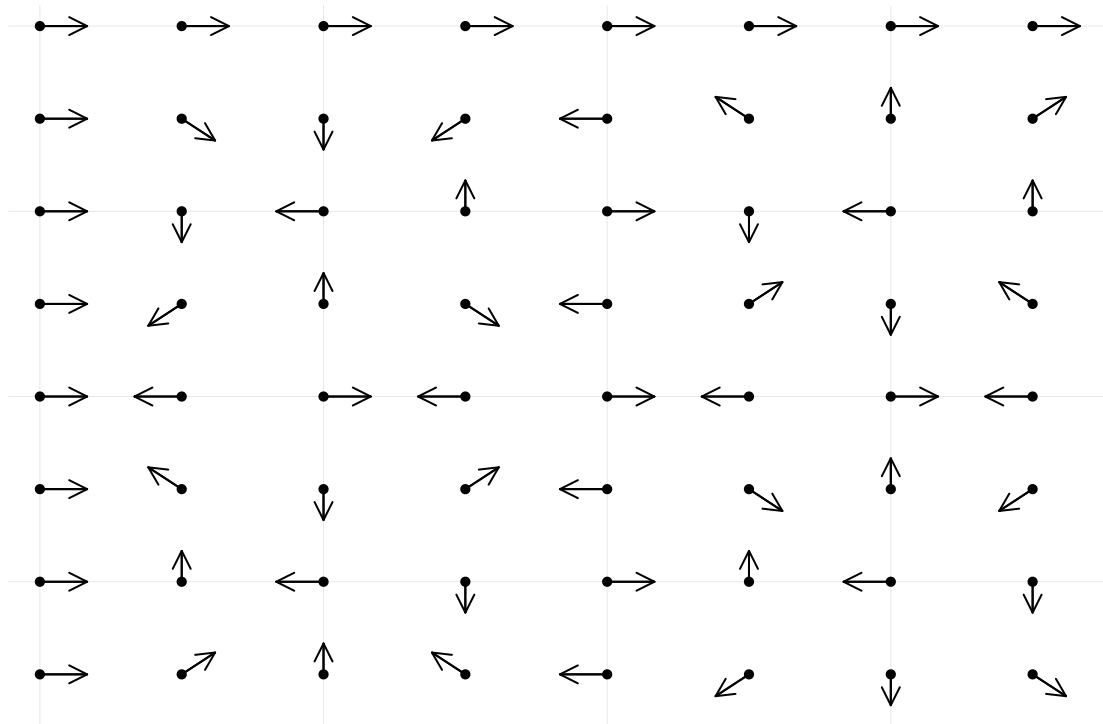
On a acquis une intuition géométrique sur les racines z_N . On va s'en servir pour visualiser cette matrice.

```
8x8 Matrix{ComplexF64}:
 1.0+0.0im    1.0+0.0im    1.0+0.0im ... 1.0+0.0im    1.0+0.0im
 1.0+0.0im    0.707107-0.707107im 0.0-1.0im 0.0+1.0im 0.707107+0.707107im
 1.0+0.0im    0.0-1.0im    -1.0-0.0im -1.0-0.0im 0.0+1.0im
 1.0+0.0im   -0.707107-0.707107im 0.0+1.0im 0.0-1.0im -0.707107+0.707107im
 1.0+0.0im    -1.0-0.0im    1.0-0.0im    1.0-0.0im   -1.0-0.0im
 1.0+0.0im   -0.707107+0.707107im 0.0-1.0im ... 0.0+1.0im -0.707107-0.707107im
 1.0+0.0im    0.0+1.0im    -1.0-0.0im -1.0-0.0im 0.0-1.0im
 1.0+0.0im    0.707107+0.707107im 0.0+1.0im 0.0-1.0im 0.707107-0.707107im
```

1 `F(8)`

► **Qu'observe-t-on dans la matrice ci-dessous ?**

Besoin d'un indice ? ☐



On peut le vérifier numériquement également:

```
partie_mauve = 4x4 Matrix{ComplexF64}:
 1.0+0.0im  1.0+0.0im  1.0+0.0im  1.0+0.0im
 1.0+0.0im  0.0-1.0im -1.0-0.0im  0.0+1.0im
 1.0+0.0im -1.0-0.0im  1.0-0.0im -1.0-0.0im
 1.0+0.0im  0.0+1.0im -1.0-0.0im  0.0-1.0im
```

```
1 partie_mauve = F(8)[1:4, 1:2:8]
```

```
partie_bleue = 4x4 Matrix{ComplexF64}:
 1.0+0.0im  1.0-0.0im  1.0-0.0im  1.0-0.0im
 1.0+0.0im  0.0-1.0im -1.0-0.0im  0.0+1.0im
 1.0+0.0im -1.0-0.0im  1.0-0.0im -1.0-0.0im
 1.0+0.0im  0.0+1.0im -1.0-0.0im  0.0-1.0im
```

```
1 partie_bleue = F(8)[5:8, 1:2:8]
```

```
true
```

```
1 partie_mauve == partie_bleue
```

```
partie_verte = 4x4 Matrix{ComplexF64}:
 1.0+0.0im      1.0+0.0im      ...      1.0+0.0im
 0.707107-0.707107im -0.707107-0.707107im  0.707107+0.707107im
 0.0-1.0im      0.0+1.0im      0.0+1.0im
-0.707107-0.707107im  0.707107-0.707107im -0.707107+0.707107im
```

```
1 partie_verte = F(8)[1:4, 2:2:8]
```

```
partie_rouge = 4x4 Matrix{ComplexF64}:
      -1.0-0.0im      -1.0-0.0im      ...      -1.0-0.0im
 -0.707107+0.707107im  0.707107+0.707107im  -0.707107-0.707107im
      0.0+1.0im      0.0-1.0im      0.0-1.0im
 0.707107+0.707107im -0.707107+0.707107im  0.707107-0.707107im
```

```
1 partie_rouge = F(8)[5:8, 2:2:8]
```

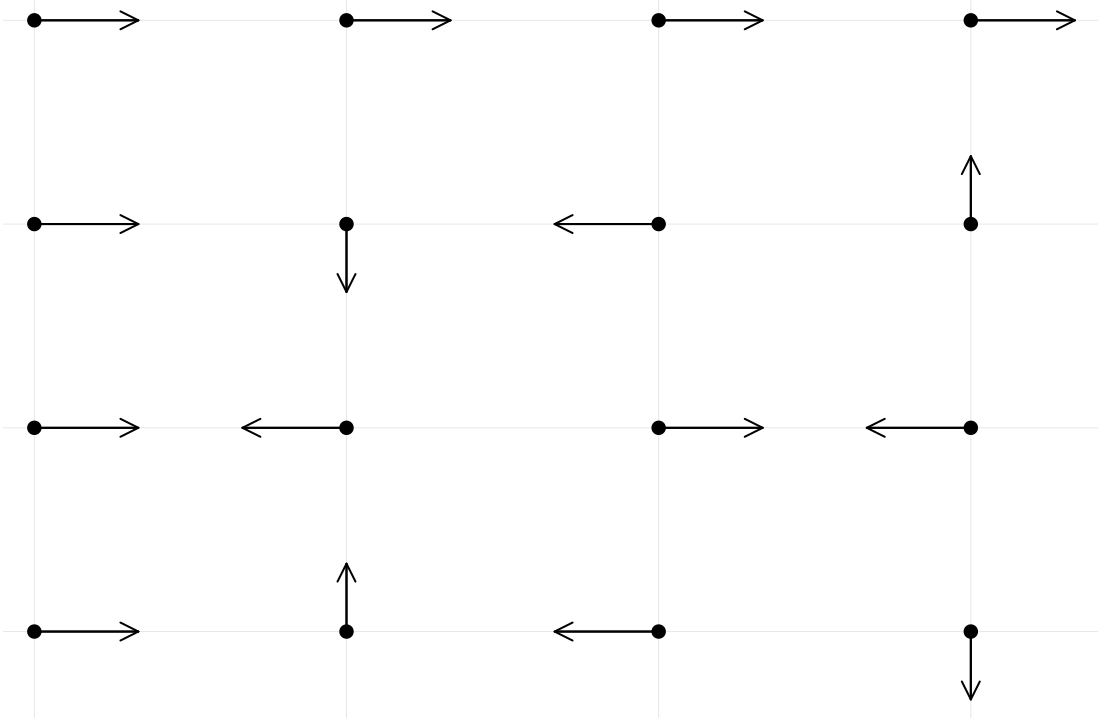
```
true
```

```
1 partie_verte == -partie_rouge
```

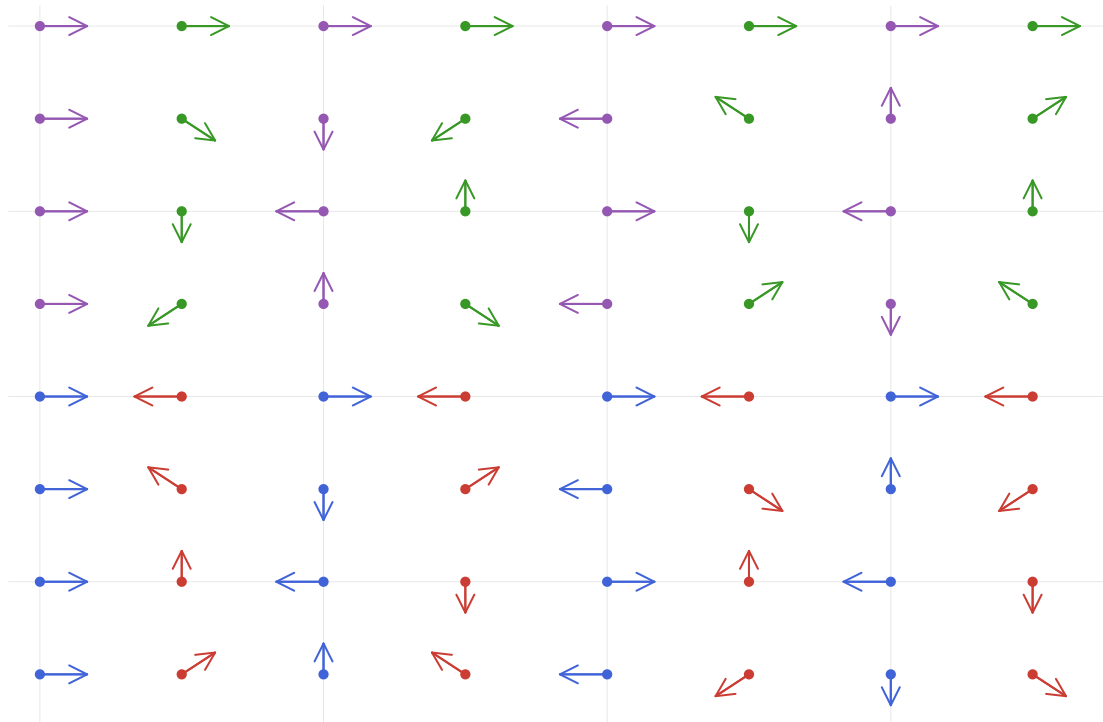
```
true
```

```
1 isapprox(0.0, 1e-300, atol = 1e-8)
```

► Qu'observe-t-on en comparant cela à la matrice 2 fois plus petite ?



```
1 arrows(4, false)
```



```
1 arrows(8, true)
```

Vérifions le à nouveau numériquement:

```
4x4 Matrix{ComplexF64}:
 1.0+0.0im  1.0-0.0im  1.0-0.0im  1.0-0.0im
 1.0+0.0im  0.0-1.0im -1.0-0.0im  0.0+1.0im
 1.0+0.0im -1.0-0.0im  1.0-0.0im -1.0-0.0im
 1.0+0.0im  0.0+1.0im -1.0-0.0im  0.0-1.0im
```

```
1 partie_bleue
```

```
4x4 Matrix{ComplexF64}:
 1.0+0.0im  1.0+0.0im  1.0+0.0im  1.0+0.0im
 1.0+0.0im  0.0-1.0im -1.0-0.0im  0.0+1.0im
 1.0+0.0im -1.0-0.0im  1.0-0.0im -1.0-0.0im
 1.0+0.0im  0.0+1.0im -1.0-0.0im  0.0-1.0im
```

```
1 F(4)
```

```
true
```

```
1 partie_bleue == F(4)
```

```
4x4 Matrix{ComplexF64}:
 1.0+0.0im      1.0+0.0im      ...      1.0+0.0im
 0.707107-0.707107im -0.707107-0.707107im      0.707107+0.707107im
 0.0-1.0im      0.0+1.0im      0.0+1.0im
-0.707107-0.707107im 0.707107-0.707107im      -0.707107+0.707107im
```

```
1 partie_vert
```

```
0.7071067811865476 - 0.7071067811865475im
```

```
1 cispi(-2/8)
```

```
► [1.0+0.0im, 0.707107-0.707107im, 2.22045e-16-1.0im, -0.707107-0.707107im]
```

```
1 cispi(-2/8) .^ collect(0:3)
```

```
4x4 Matrix{ComplexF64}:
 1.0+0.0im      1.0+0.0im      ...      1.0+0.0im
 0.707107-0.707107im -0.707107-0.707107im      0.707107+0.707107im
 2.22045e-16-1.0im -2.22045e-16+1.0im      -2.22045e-16+1.0im
-0.707107-0.707107im 0.707107-0.707107im      -0.707107+0.707107im
```

```
1 cispi(-2/8) .^ (0:3) .* F(4)
```

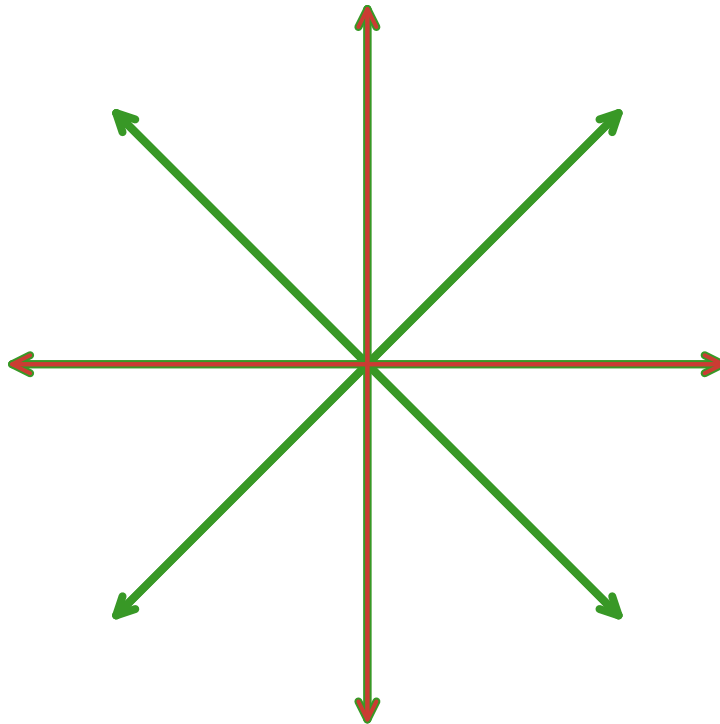
```
4x4 BitMatrix:
```

```
1 1 1 1
1 1 1 1
1 1 1 1
1 1 1 1
```

```
1 isapprox.(real.(partie_vert), real.(cispi(-2/8) .^ (0:3) .* F(4)), atol=1e-8)
```

En se rappelant que l'intuition géométrique des N racines de l'unité, on comprend pourquoi les $N/2$ racines sont un sous-ensembles des N racines.

max_N = 8



► Quel est la complexité de l'algorithme FFT.

Utilitaires ⇄

.....

```
1 using Plots
```

```
1 using PlutoUI, FFTW, Images, TestImages, ImageFiltering
```

```
1 using Polynomials, LinearAlgebra
```

```
1 import WAV
```

```
play_sound (generic function with 1 method)
```

```
1 # See https://discourse.julialang.org/t/how-do-i-play-sound-in-a-notebook-preferably-pluto/49403/8?u=blegat
2 function play_sound(sound, samplerate)
3     dir = mktempdir(cleanup = false)
4     file = joinpath(dir, "audio.wav")
5     WAV.wavwrite(Int.(trunc.(real.(sound*2^15))), file, Fs=samplerate, nbits=16)
6     md"""$(LocalResource(file))"""
7 end
```

```

1 begin
2   struct Join
3     list
4     Join(a) = new(a)
5     Join(a, b, args...) = Join(tuple(a, b, args...))
6   end
7   function Base.show(io::IO, mime::MIME"text/html", d::Join)
8     for el in d.list
9       show(io, mime, el)
10    end
11  end
12 end

```

```

1 begin
2   struct HTMLTag
3     tag::String
4     parent
5   end
6   function Base.show(io::IO, mime::MIME"text/html", d::HTMLTag)
7     write(io, "<", d.tag, ">")
8     show(io, mime, d.parent)
9     write(io, "</", d.tag, ">")
10  end
11 end

```

f (generic function with 1 method)

```
1 f(i, j, n) = cispi(-2 * i * j / n)
```

F (generic function with 1 method)

```

1 function F(n)
2   A = [
3     f(k, m, n)
4     for k in 0:(n-1), m in 0:(n-1)
5   ]
6 end

```

8×8 Matrix{ComplexF64}:

```

1.0+0.0im    1.0+0.0im    1.0+0.0im    ...    1.0+0.0im    1.0+0.0im
1.0+0.0im    0.707107-0.707107im  0.0-1.0im    0.0+1.0im    0.707107+0.707107im
1.0+0.0im    0.0-1.0im    -1.0-0.0im   -1.0-0.0im    0.0+1.0im
1.0+0.0im   -0.707107-0.707107im  0.0+1.0im    0.0-1.0im   -0.707107+0.707107im
1.0+0.0im    -1.0-0.0im    1.0-0.0im    1.0-0.0im    -1.0-0.0im
1.0+0.0im   -0.707107+0.707107im  0.0-1.0im    ...    0.0+1.0im   -0.707107-0.707107im
1.0+0.0im    0.0+1.0im    -1.0-0.0im   -1.0-0.0im    0.0-1.0im
1.0+0.0im    0.707107+0.707107im  0.0+1.0im    0.0-1.0im    0.707107-0.707107im

```

```
1 F(8)
```

arrows (generic function with 2 methods)

```
1 function arrows(n, use_color = false)
2     plot(showaxis = false)
3     for i in 0:(n-1)
4         for j in 0:(n-1)
5             if use_color
6                 color = Colors.JULIA_LOGO_COLORS[1 + iseven(j) * 2 + (2i < n)]
7             else
8                 color = :black
9             end
10            scatter!([j], [-i], markersize = 8 / sqrt(n), markerstrokewidth = 0,
11                    legend = nothing; color)
12            c = f(i, j, n) / 3 # Divide by 3 so that arrow is not too long in plot
13            plot!([j, j + real(c)], [-i, -i + imag(c)], arrow = true, legend =
14                  nothing; color)
15        end
16    end
17    plot!()
```

draw_roots! (generic function with 1 method)

```
1 function draw_roots!(N; kws...)
2     for i in 1:N
3         b, a = sincospi(2 * i / N)
4         plot!([0, a], [0, b]; arrow = true, label = nothing, kws...)
5     end
6 end
```

qa (generic function with 1 method)

```
1 function qa(question, answer)
2     return HTMLTag("details", Join(HTMLTag("summary", question), answer))
3 end
```